INSIDE STLC ACADEMY

API TESTING MASTERCLASS

API Test Strategy Template

A reusable, fill-in strategy for testing any API.

A test strategy is the one-page (or few-page) document that says what you will test, how, where and when you will stop. This is a genuine fill-in template: copy it, replace every prompt in [square brackets] with your project's detail, and delete the guidance you no longer need. Aim for clarity over length — a strategy nobody reads is worthless.

How to Use This Template

1. Copy the sections below into your project doc or wiki.
2. Answer every prompt — if a section genuinely does not apply, write "N/A" and say why rather than deleting it.
3. Keep it living: update it when scope, tools or environments change.
4. Review it with a developer and a product owner before you start — a strategy is an agreement, not a solo essay.

1. Scope & Objectives

State what this effort covers and, just as importantly, what it does not. Objectives should be outcomes, not activities.

- API(s) / services under test: [name, version, base URLs]
- In scope: [endpoints, features, flows]
- Out of scope: [e.g. third-party APIs, UI, performance] and why
- Objectives: [e.g. verify all documented endpoints meet their contract; confirm access control holds]

2. Risks

List what could go wrong and how badly, so testing effort follows the risk. Rank so the team can focus.

Risk	Likelihood (H/M/L)	Impact (H/M/L)	Mitigation / test focus
[e.g. Broken access control exposes other users' data]	[ ]	[ ]	[ ]
[e.g. Breaking change to response schema]	[ ]	[ ]	[ ]
[add rows as needed]	[ ]	[ ]	[ ]

3. Test Types & Coverage

Which kinds of testing will you do, and to what depth? Tick, delete or expand as appropriate.

- Functional (happy path + negative/error responses): [depth]
- Contract / schema validation against the spec: [yes/no, how]
- Authentication & authorisation / access control: [scenarios]
- Data validation & boundaries (types, required, limits): [depth]
- Non-functional — performance / load / security: [in scope? owned by whom?]
- Integration between services / end-to-end flows: [which flows]

4. Environments

Where will testing run, and what state can you rely on there?

Environment	Base URL	Purpose	Notes / limitations
Dev	[url]	[early integration]	[unstable data]
Staging / QA	[url]	[main test target]	[prod-like]
Production	[url]	[smoke only]	[read-only? which checks are safe?]

5. Test Data

Define how you obtain, create and clean up data, and how you protect anything sensitive.

- Source: [seeded fixtures / created via setup requests / anonymised copy]
- Accounts & roles needed: [admin, standard user, unauthorised user]
- Sensitive data handling: [no real PII; secrets stored where]
- Cleanup / teardown approach: [how created data is removed]

6. Tools

- Manual / exploratory: [Postman, curl]
- Automation & CI: [Newman, Playwright API, REST-assured, pytest]
- Reporting / tracking: [where results and defects go]
- Spec / contract source: [OpenAPI location]

7. Entry & Exit Criteria

Entry criteria (before testing starts)

- [Spec / documentation available and stable]
- [Test environment deployed and reachable]
- [Credentials and test accounts provisioned]

Exit criteria (when testing is "done")

- [All planned test cases executed]
- [No open Critical/High defects; agreed threshold for Medium/Low]
- [Contract checks pass against the current spec]
- [Results reported and signed off]

8. Reporting

Say what you will report, to whom, and how often, so results are visible and defects are actionable.

- Cadence & audience: [e.g. daily summary to the team, sign-off report to PO]
- Metrics: [pass/fail counts, coverage of endpoints, open defects by severity]
- Defect process: [where raised, required detail, severity definitions]

9. Sign-off

Record who agrees the strategy and who accepts the results — a strategy without owners is easy to ignore.

Role	Name	Date	Signature / approval
QA / Test lead	[ ]	[ ]	[ ]
Development lead	[ ]	[ ]	[ ]
Product owner	[ ]	[ ]	[ ]

> **INSIDE STLC PRO TIP**
>
> Keep this to a page or two and treat it as living. A concise, agreed strategy that the whole team has read beats a beautiful 20-page document nobody opens. Bring a completed version of this template to an interview — it shows you think strategically, not just click "Send".